Facens

# Disciplina

**Algoritmos e Programação**

# Conteúdo

**Estruturas de Repetição**
**Comandos** *while* **e** *do-while*

Ao final desta aula o estudante deverá
**Aplicar estruturas de repetição while e do-while em algoritmos desenvolvidos em Linguagem C**



Facens
AQUI TEM ENGENHARIA
material didático

# Estruturas de repetição

- Estruturas de repetição são também chamados de laços.
- Os **laços** são comandos usados sempre que uma ou mais instruções tiverem de ser repetidas enquanto uma certa condição estiver sendo satisfeita.
- Em C existem três comandos de laços:

  - for;
  - **while**;
  - **do-while**.

Facens
AQUI TEM ENGENHARIA
material didático

# Comando **while**

O comando **while** é utilizado quando **não se conhece** a quantidade de repetições que irão ocorrer durante a execução do programa. O **while** executa um ou mais comandos **enquanto** uma determinada condição permanecer verdadeira.

**Sintaxe:**

```
while (condição)
    comando;
```

1 único comando

```
while (condição)
 {
    comando;
    comando;
 }
```

bloco de comandos

# Comando **while**

**Como funciona:**

[1] É testada a **condição**

Se a condição for **verdadeira:**

[2] executa-se o **bloco de comandos** e volta para o [1]

Se a condição for **falsa:**

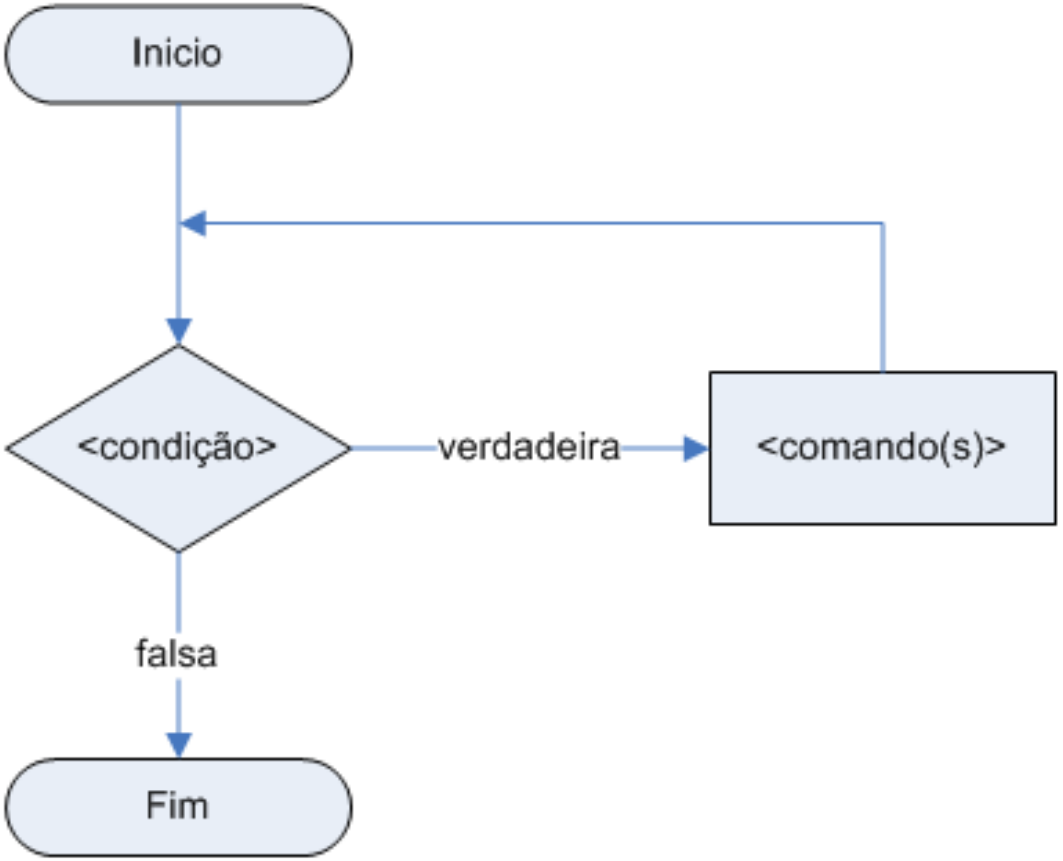
[3] o loop é encerrado.

```
            1
while (condição)
 {
   comando;
   comando;   2
 }
```

3

# Comando **while**

- Representação Esquemática




Facens
AQUI TEM ENGENHARIA
material didático

# Comando **while**

- O teste é avaliado **antes** do(s) comando(s) ser(em) executado(s).
- Se na 1ª vez, a condição for **falsa**, não será executado nenhum comando.
- A **condição** pode ser qualquer expressão válida em C.
- A estrutura de repetição **while** pode ser usada de forma aninhada.
- A estrutura de repetição **for** pode ser substituída por um **while**, porém o inverso não é verdadeiro.

Facens
AQUI TEM ENGENHARIA
material didático

# Comando **while**

- **Exemplo: Exibir 10 números inteiros**

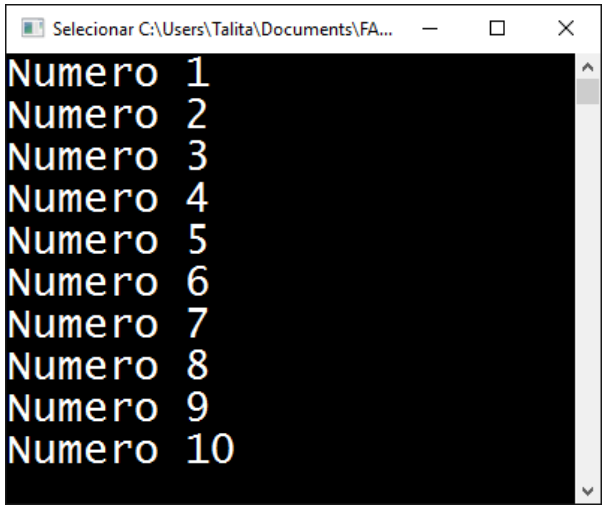
```
main()
{
int i;

for(i=1;i<=10;i++)
  printf("\nNumero %i",i);

getch();
return 0;
}
```

```
main()
{
int i=1;

while(i<=10)
  {
    printf("\nNumero %i",i);
    i++;
   }

getch();
return 0;
}
```

# Comando **while**

**Exemplo:** Exibir 10 números inteiros (**while** usado como **for**).

```
main()
{
    int i = 1;
    while (i<=10)
    {
      printf("\nNumero %i",i);
      i++;
    }
    return 0;
}
```




Facens
AQUI TEM ENGENHARIA
material didático

# Comando **do-while**

Como o **while**, o **do-while** também é utilizado quando **não se conhece** a quantidade de repetições que irão ocorrer durante a execução do programa. O **do-while** executa um ou mais comandos **enquanto** uma determinada condição permanecer verdadeira.

- O comando **do-while** é uma variação do while, cuja **diferença** é que a **condição é executada após o laço**, ou seja, **pelo menos uma vez as instruções são executadas** e só depois é verificada a condição de teste.

**Sintaxe:**

```
do
 {
    comando;
    comando;
 } while (condição) ;
```

1 único comando

bloco de comandos

# Comando **do-while**

**Como funciona:**

[1] executa-se o **bloco de comandos**

[2] testa-se a **condição**

Se a condição for **verdadeira:**

volta para o **[1]**

Se a condição for **falsa:**

**[3]** o loop é encerrado.

```
do
 {
   comando;
   comando;
 } while (condição) ;
```

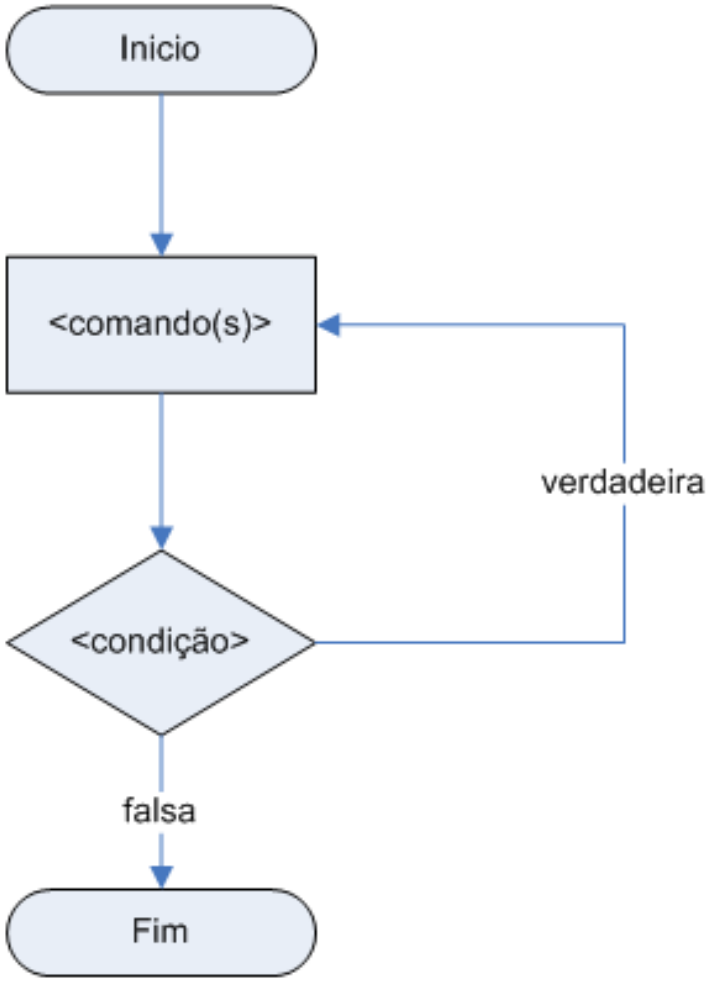
1

2

3

# Comando **do-while**

- Representação Esquemática




Facens
AQUI TEM ENGENHARIA
material didático

# Comando **do-while**

- O teste é avaliado **após** o(s) comando(s) ser(em) executado(s).
- Os comandos serão executados 1 vez, mesmo se a condição for **falsa**.
- A **condição** pode ser qualquer expressão válida em C.
- A estrutura de repetição **do-while** também pode ser usada de forma aninhada.
- A estrutura de repetição **for** pode ser substituída por um **do-while**, porém o inverso não é verdadeiro.

# Comando **do-while**

- **Exemplo: Exibir 10 números inteiros**

```
main()
{
int i;

for(i=1;i<=10;i++)
  printf("\nNumero %i",i);

getch();
return 0;
}
```

```
main()
{
int i=1;

do
 {
   printf("\nNumero %i",i);
   i++;
  } while(i<=10);

getch();
return 0;
}
```

# Comando **do-while**

- Apesar das instruções **while** e **do-while** parecerem idênticas, elas **não** são.

**Exemplo:**

```
main()
{
int i=-10;

while(i>0)
 {
   printf("\nNumero %i",i);
   i++;
  }
getch();
return 0;
}
```

```
Não irá exibir NADA
```

```
main()
{
int i=-10;

do
 {
   printf("\nNumero %i",i);
   i++;
  } while(i>0);
getch();
return 0;
}
```

```
Irá exibir 1 vez
```

# Comando **do-while**

- O **do-while** é usado, portanto, em situações em que se tenha um **menu** de opções ou mesmo quando ainda **não se conhece o valor da variável** que será testada, sendo assim é preciso recebê-la primeiro e depois verificar (testar) o seu conteúdo.

# Resumo

**Estruturas de Repetição**

**while**

- Utilizado quando **não conhece a quantidade de repetições** (**o número de vezes**)
- Executa 1 ou mais comandos **enquanto** a condição for **verdadeira**

```
while (condição)
 {
    comando;
    comando;
 }
```

**Comando**

**do-while**

- Utilizado quando **não conhece a quantidade de repetições** (**o número de vezes**)
- Executa 1 ou mais comandos **enquanto** a condição for **verdadeira.** **Diferença:** a **condição é executada após o laço**, ou seja, **pelo menos 1 vez as instruções são executadas** e só depois é verificada a condição de teste

```
do
 {
    comando;
    comando;
 } while (condição) ;
```

Facens
AQUI TEM ENGENHARIA
material didático

# Comando **while**

**Exemplo:** Calcular e exibir a área e o perímetro de um quadrado. Repetir até o usuário digitar [N]ão.

```
main()
{
    char op='s';
    float lado,p,a;
    while (toupper(op) == 'S')//Biblioteca <ctype.h>
    {
        printf("\nDigite um lado de um quadrado: ");
        scanf("%f",&lado);
        a = pow(lado,2);
        p = 4*lado;
        printf("\nArea:%.2f\nPerimetro: %.2f",a,p);
        printf("\nDeseja continuar?[S]Sim ou [N]Não:");
        fflush(stdin); //força receber caracteres...
        scanf("%c",&op); //...ou faça " %c"
    }
    system("pause");
    return 0;
}
```

Facens
AQUI TEM ENGENHARIA
material didático

# Execução do **while**

O funcionamento do comando while segue os seguintes passos:

1. A condição é avaliada (`toupper(op) == 'S'`)
2. Se a condição for **falsa**, o laço termina e executa o comando que está logo após o bloco subordinado ao while (`system("pause");`)
3. Se condição for **verdadeira** então executa os comandos que estão subordinados ao while;
4. Volta ao passo 1.

Facens
AQUI TEM ENGENHARIA
material didático

# Comando **while**

- **Exercício 1:**
  - Verifique os erros e reescreva o código abaixo:

    Dado um conjunto de números inteiros, calcular os seus quadrados. O programa é interrompido quando o número digitado for igual a 0.

```
main ()
{
   int num, quad;
   printf("Digite um numero:\n");
   scanf("%f", &num);
   while (num == 0)
   {
       quad = pow(num,num);
       printf ("O quadrado de %d = %d\n", num, quad);
       scanf("%d", &quad);
   }
   system ("PAUSE");
   return 0;
}
```

Facens
AQUI TEM ENGENHARIA
material didático

# Comando **while**

- **Exercício 1:**

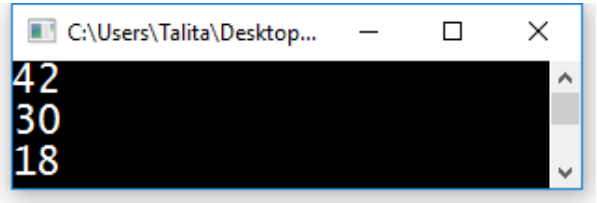
```
main ()
{
  int num, quad;
  printf("Digite um numero:\n");
  scanf("%i", &num);
  while (num != 0)
  {
      quad = pow(num,2);
      printf ("O quadrado de %i = %i\n", num, quad);
      scanf("%i", &num);
  }
  system ("PAUSE");
  return 0;
}
```

Facens
AQUI TEM ENGENHARIA
material didático

# Comando **while**

- **Exercício 2:**
  - Observe o programa abaixo e escreva qual é a sua saída.

```
#include<stdio.h>
#include<conio.h>
int main()
{
    int a=1,b=20;

    while(a<=137 && b>=0)
    {
        printf("%i\n",(a+b)*2);
        a+=4; //a=a+4
        b-=10; //b=b-10
    }
    getch();
    return 0;
}
```




Facens
AQUI TEM ENGENHARIA
material didático

# Comando **do-while**

- **Exemplo:** Calcular o triplo de um número e se número for igual a zero finaliza a execução.

```
main()
{
    int n;
    do
    {
        printf("\nDigite um numero ou zero para sair: ");
        scanf("%i",&n);
        printf("O triplo do numero %i \x82 %i\n",n,n*3);
    }while (n != 0);

    getch();
    return 0;
}
```

Facens
AQUI TEM ENGENHARIA
material didático

# Comando **do-while**

- **Exemplo:** Evitar que o usuário digite um valor inválido.

```
main()
{
    int op;
    do
    {
        printf("\n Escolha uma ação pelo código: ");
        printf("\n1-Debito\n2-Credito\n3-Cancelar: ");
        scanf("%i",&op);
    }while (op<1 || op>3);
    //Realizar operações específicas para a opção escolhida

    getch();
    return 0;
}
```

Facens
AQUI TEM ENGENHARIA
material didático

# Execução do **do-while**

- O funcionamento do comando do-while segue os seguintes passos:

1. A instrução ou bloco de instruções é executada;
2. A condição é avaliada (`op<1 || op>3`);
3. Se condição for **verdadeira**,volta-se ao passo 1.
4. Se a condição for **falsa**, o laço termina e executa o comando que está logo após o bloco subordinado ao while (`getch()`);

Facens
AQUI TEM ENGENHARIA
material didático

# Comando **do-while**

- **Exercício 1:**
  - Receber 2 notas de um aluno e calcular a média final. O usuário só pode digitar notas no intervalo de 0 a 10, caso ele digite um valor inválido mostre uma mensagem e solicite uma nova nota.

Facens
AQUI TEM ENGENHARIA
material didático

# Comando **do-while**

- **Exercício 1:**

```
main()
{
    float n1,n2;
    do
    {
       printf("Digite a nota 1: ");
       scanf("%f",&n1);
       printf("Digite a nota 2: ");
       scanf("%f",&n2);
       if (n1<0 || n1>10 || n2<0 || n2>10)
       printf("Valor invalido!\n\n");
    }while(n1<0 || n1>10 || n2<0 || n2>10);
    printf("\nMedia = %.2f",(n1+n2)/2);
    getch();
    return 0;
}
```

Facens
AQUI TEM ENGENHARIA
material didático

# Comando **do-while**

- **Exercício 2:**
  - Melhore o exercício 1 e faça para a quantidade indeterminada de alunos. Pergunte ao usuário se ele deseja continuar calculando as médias. Faça até o usuário digitar ‘N’, **não** deseja continuar.

# Comando **do-while**

- **Exercício 3:**
  - Criar um programa capaz de somar um número indeterminado de valores inteiros positivos.
  - Caso um número digitado seja negativo mostre uma mensagem informando que o número é inválido.
  - Para cada novo número inserido no programa pergunte se o usuário deseja continuar inserindo os números. Finalize a entrada de dados quando o usuário digitar 'N', o que corresponde a opção de **não** continuar a receber números.

Facens
AQUI TEM ENGENHARIA
material didático

# Comando **do-while**

```
#include<stdio.h>
#include<conio.h>
#include<ctype.h>
main()
{
    int valor, soma=0;
    char resp;
    do
    {
        do
        {
            printf("Digite um valor: ");
            scanf("%i", &valor);
            if(valor<0)
            printf("\nValor invalido! \n");
        }while(valor<0);
     soma+=valor;
     printf("\nDeseja continuar?\n[S]Sim\n[N]Nao\n");
     scanf(" %c",&resp);
     }while (toupper(resp) != 'N');
printf("\nSoma= %i",soma);
}
```

Facens
AQUI TEM ENGENHARIA
material didático

# Comando **do-while**

- **Exercício 3: Continuação - Validação da Resposta**

```
do
{
      do
      {
         printf("\nDeseja continuar?\n[S]Sim\n[N]Nao\n");
         scanf(" %c",&resp);
         resp=toupper(resp); //caracter maiúsculo
         if (resp!='N' && resp!='S')
         {
             printf("Por favor, digite S-sim e N-Não\n");
         }
      }while (resp!='N' && resp!='S');
  }while (resp != 'N');
  printf("\nSoma= %i",soma);
  getch();
  return 0;
}
```

Facens
AQUI TEM ENGENHARIA
material didático

# Comando **do-while**

- **Exercício 4:**
  - Elabore um programa que solicite um número inteiro ao usuário e crie um novo número inteiro com os dígitos em ordem inversa. Caso o número digitado seja 0 retorne o valor 0. Abaixo o exemplo de uma execução do programa:

```
Digite um número inteiro:     5382
Seu número invertido é:       2835
```

Facens
AQUI TEM ENGENHARIA
material didático

# Comando **do-while**

- **Exercício 4:**

```
int numero, resto;
printf("Digite um numero inteiro: ");
scanf ("%i", &numero);
printf("\nSeu numero invertido \x82: ");
if(numero==0)
{
        printf ("%i",numero);
}
while(numero>0)
{
       resto=numero%10;
       printf("%i",resto);
       numero=numero/10;
}
```

Entrada: 245
Saída: ?

245|_10
45 24
5
Resto=**5**
Numero=245/10=24

24|_10
4 2
Resto=**4**
Numero=24/10=2

2|_10
2 0

Resto=**2**
Numero=2/10=0

Facens
AQUI TEM ENGENHARIA
material didático

# Comando **while** e **do-while**

- **Exercício 5:**
  - Escreva um programa para receber o **saldo inicial** de uma conta bancária de um cliente.
  - A seguir realizar um número indeterminado de operações de depósito e retirada.
  - Selecionar respectivamente o tipo da operação, sendo: **1.Depósito 2.Retirada 3.Fim**, e também o **valor**.
  - Se o número da operação for inválido, mostrar uma mensagem de erro e voltar para o menu.
  - Ao informar a opção 3, o programa deve ser encerrado e mostrar o saldo final da conta com as seguintes mensagens:

    **Conta zerada** (saldo zero), **conta estourada** (se o saldo for negativo) ou **conta preferencial** (se o saldo for positivo).